

main.c	Share Run	Output
<pre>1 // 4. Convert an infix expression to postfix using a stack 2 #include <stdio.h> 3 #include <stdlib.h> 4 #include <string.h> 5 #include <ctype.h> 6 7 #define MAX 100 8 9 char stack[MAX]; 10 int top = -1; 11 12 void push(char c) { 13 if (top >= MAX - 1) { 14 printf("Stack Overflow\n"); 15 return; 16 } 17 stack[++top] = c; 18 } 19 20 char pop() { 21 if (top < 0) {</pre>	<pre>Enter an infix expression: (A+B)*C Postfix expression: AB+C* === Code Execution Successful ===</pre>	

```
22     return '\0';
23 }
24 return stack[top--];
25 }
26
27 char peek() {
28     if (top < 0) {
29         return '\0';
30     }
31     return stack[top];
32 }
33
34 int precedence(char op) {
35     switch (op) {
36         case '+':
37         case '-':
38             return 1;
39         case '*':
40         case '/':
41             return 2;
42         case '^':
```

Enter an infix expression: (A+B)*C
Postfix expression: AB+C*

=== Code Execution Successful ===

```

43         return 3;
44     default:
45         return 0;
46     }
47 }
48 int isOperator(char c) {
49     return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^'
50         );
51 }
52 void infixToPostfix(char *infix, char *postfix) {
53     int i = 0, j = 0;
54     char c;
55     while ((c = infix[i++]) != '\0') {
56         // If operand, add to output
57         if (isalnum(c)) {
58             postfix[j++] = c;
59         }
60         // If '(', push to stack
61         else if (c == '(') {
62             push(c);
63         }
64     }

```

Enter an infix expression: (A+B)*C
Postfix expression: AB+C*

=== Code Execution Successful ===

```

63 ~ // If ')', pop until '(' is found
64 ~ else if (c == ')') {
65 ~     while (top >= 0 && peek() != '(') {
66 ~         postfix[j++] = pop();
67 ~     }
68 ~     pop(); // Remove '(' from stack
69 ~ }
70 ~ // If operator
71 ~ else if (isOperator(c)) {
72 ~     // Handle right-associativity for '^'
73 ~     if (c == '^') {
74 ~         while (top >= 0 && precedence(peek()) > precedence(c)) {
75 ~             postfix[j++] = pop();
76 ~         }
77 ~     } else {
78 ~         while (top >= 0 && precedence(peek()) >= precedence(c)) {
79 ~             postfix[j++] = pop();
80 ~         }
81 ~     }

```

Enter an infix expression: (A+B)*C
Postfix expression: AB+C*

=== Code Execution Successful ===

```
83     }
84 }
85 // Pop remaining operators
86 while (top >= 0) {
87     postfix[j++] = pop();
88 }
89 postfix[j] = '\0';
90 }
91 int main() {
92     char infix[MAX], postfix[MAX];
93     printf("Enter an infix expression: ");
94     scanf("%s", infix);
95     infixToPostfix(infix, postfix);
96     printf("Postfix expression: %s\n", postfix);
97     return 0;
98 }
```

=== Code Execution Successful ===

```
1 // 5.evaluate a postfix expression using a stack
2 #include <stdio.h>
3 #include <ctype.h>
4
5 #define MAX 100
6
7 int stack[MAX];
8 int top = -1;
9
10 // Push operation
11 void push(int value)
12 {
13     top++;
14     stack[top] = value;
15 }
16
17 // Pop operation
18 int pop()
19 {
20     return stack[top--];
21 }
```

Enter postfix expression: 23*54*+9-
Result = 17

=== Code Execution Successful ===

```

22
23 int main()
24 {
25     char postfix[MAX];
26     int i, op1, op2, result;
27
28     printf("Enter postfix expression: ");
29     scanf("%s", postfix);
30
31     for(i = 0; postfix[i] != '\0'; i++)
32     {
33         // If character is operand
34         if(isdigit(postfix[i]))
35         {
36             push(postfix[i] - '0');
37         }
38         // If character is operator
39         else
40         {
41             op2 = pop();
42             op1 = pop();

```

Enter postfix expression: 23*54*+9-
Result = 17

=== Code Execution Successful ===

```
44         switch(postfix[i])
45     {
46         case '+':
47             result = op1 + op2;
48             break;
49
50         case '-':
51             result = op1 - op2;
52             break;
53
54         case '*':
55             result = op1 * op2;
56             break;
57
58         case '/':
59             result = op1 / op2;
60             break;
61
62         case '%':
63             result = op1 % op2;
64             break;
```

Enter postfix expression: 23*54*+9-
Result = 17

=== Code Execution Successful ===


```
65
66         default:
67             printf("Invalid operator");
68             return 1;
69     }
70
71     push(result);
72 }
73 }
74
75 printf("Result = %d", pop());
76
77 return 0;
78 }
79
```

main.c	   Share 	Output
<pre>1 // 6. Implement queue operations using arrays and linked list 2 // 1 Queue operations using arrays 3 #include <stdio.h> 4 #define MAX 5 5 6 int queue[MAX]; 7 int front = -1, rear = -1; 8 9 // Insert element 10 void enqueue(int value) 11 { 12 if(rear == MAX - 1) 13 { 14 printf("Queue Overflow\n"); 15 } 16 else 17 { 18 if(front == -1) 19 front = 0; 20 21 rear++;</pre>	<pre>10 inserted 20 inserted 30 inserted Queue Elements: 10 20 30 10 deleted Queue Elements: 20 30 === Code Execution Successful ===</pre>	

main.c		Run	Output
22	queue[rear] = value;		10 inserted
23			20 inserted
24	printf("%d inserted\n", value);		30 inserted
25	}		Queue Elements:
26	}		10 20 30
27			10 deleted
28	// Delete element		Queue Elements:
29	void dequeue()		20 30
30	{		
31	if(front == -1 front > rear)		
32	{		
33	printf("Queue Underflow\n");		
34	}		
35	else		
36	{		
37	printf("%d deleted\n", queue[front]);		
38	front++;		
39	}		
40	}		

=== Code Execution Successful ===

```
42 // Display queue
43 void display()
44 {
45     int i;
46
47     if(front == -1 || front > rear)
48     {
49         printf("Queue is Empty\n");
50     }
51     else
52     {
53         printf("Queue Elements:\n");
54
55         for(i = front; i <= rear; i++)
56         {
57             printf("%d ", queue[i]);
58         }
59     }
60
61     printf("\n");
62 }
```

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

=== Code Execution Successful ===
```

```
60
61     printf("\n");
62 }
63
64 int main()
65 {
66     enqueue(10);
67     enqueue(20);
68     enqueue(30);
69
70     display();
71
72     dequeue();
73
74     display();
75
76     return 0;
77 }
```

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
```

=== Code Execution Successful ===

main.c

Share

Run

```
1 // 6. Implement queue operations using arrays and linked list
2 // 1 Queue operations using linked list
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 // Node structure
7 struct Node
8 {
9     int data;
10    struct Node *next;
11 };
12
13 struct Node *front = NULL;
14 struct Node *rear = NULL;
15
16 // Insert element
17 void enqueue(int value)
18 {
19     struct Node *newNode;
20
21     newNode = (struct Node*)malloc(sizeof(struct Node));
```

Output

10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

=== Code Execution Successful ===

main.c		Output
22		10 inserted
23	newNode->data = value;	20 inserted
24	newNode->next = NULL;	30 inserted
25		Queue Elements:
26	if(rear == NULL)	10 20 30
27	{	10 deleted
28	front = rear = newNode;	Queue Elements:
29	}	20 30
30	else	
31	{	
32	rear->next = newNode;	
33	rear = newNode;	
34	}	
35		
36	printf("%d inserted\n", value);	
37	}	
38		
39	// Delete element	
40	void dequeue()	
41	{	
42	struct Node *temp;	
		=== Code Execution Successful ===

main.c		Run	Output
43			10 inserted
44	if(front == NULL)		20 inserted
45	{		30 inserted
46	printf("Queue Underflow\n");		Queue Elements:
47	}		10 20 30
48	else		10 deleted
49	{		Queue Elements:
50	temp = front;		20 30
51			
52	printf("%d deleted\n", front->data);		
53			=== Code Execution Successful ===
54	front = front->next;		
55			
56	if(front == NULL)		
57	rear = NULL;		
58			
59	free(temp);		
60	}		
61	}		
62			
63	// Display queue		
64	void display()		


```

64 void display()
65 {
66     struct Node *temp;
67
68     if(front == NULL)
69     {
70         printf("Queue is Empty\n");
71     }
72     else
73     {
74         temp = front;
75
76         printf("Queue Elements:\n");
77
78         while(temp != NULL)
79         {
80             printf("%d ", temp->data);
81             temp = temp->next;
82         }
83     }
84 }

```

```

10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

=== Code Execution Successful ===

```

```
83     },
84
85     printf("\n");
86 }
87
88 int main()
89 {
90     enqueue(10);
91     enqueue(20);
92     enqueue(30);
93
94     display();
95
96     dequeue();
97
98     display();
99
100     return 0;
101 }
```

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

=== Code Execution Successful ===
```